المحور الأول: الفهم الأساسي والشامل

1. ما هي دورة حياة تطوير البرمجيات (SDLC)؟

تعريف:
هي إطار عمل **يصف المراحل** (Software Development Life Cycle - SDLC) دورة حياة تطوير البرمجيات **المختلفة** التي تمر بها عملية تطوير أي نظام أو تطبيق برمجي، بدءًا من **تحديد المتطلبات** وحتى **نشر النظام وصيانته**.
. وهي تمثل **خطة منظمة ومنهجية** تضمن تطوير برامج عالية الجودة بطريقة فعالة ومنخفضة المخاطر

أهمية SDLC:

ضمان جودة المنتج النهائي: من خلال اتباع خطوات مدروسة للفحص والتأكد من مطابقة البرنامج للمتطلبات

.**تقليل التكاليف والمخاطر:** بتخطيط مسبق يقلل من الأخطاء والمشكلات أثناء التطوير أو بعد الإطلاق

.**تحسين التواصل داخل الفريق:** من خلال تحديد واضح للأدوار والمسؤوليات في كل مرحلة

.**تسهيل إدارة الوقت والمشاريع:** لأنها توفر جدولة زمنية وتقديرًا أدق للموارد المطلوبة

.**القدرة على التنبؤ بالنتائج:** مما يساعد في تقييم مدى التقدم نحو الأهداف المطلوبة

سبب استخدام SDLC في مشاريع البرمجيات الحديثة:

.المشاريع الحديثة أصبحت **معقدة وكبيرة الحجم** وتحتاج إلى منهج منظم

.وجود فرق عمل متعددة (محللون، مطورون، مختبرون...) يتطلب **تنسيقًا منهجيًا**

.التغير المستمر في **احتياجات المستخدمين والسوق** يجعل وجود دورة واضحة لتحديث النظام أمرًا ضروريًا

.ضمن إطار منظم ومفهوم DevOps أو Agile تساعد على **تطبيق منهجيات حديثة** مثل

2. مراحل SDLC والمدخلات والمخرجات والتحديات في كل مرحلة:

المرحلة	المدخلات الأساسية	المخرجات الأساسية	التحديات الرئيسية
1. تحليل المتطلبات (Requirements Analysis)	- طلب العميل - وثائق الأعمال - مقابلات واستبيانات	- وثيقة المتطلبات الوظيفية (SRS) - تحليل النطاق	1. صعوبة فهم احتياجات العميل بدقة. 2. تغيّر المتطلبات أثناء المشروع. 3. ضعف التواصل بين المحللين والمستخدمين.
2. التصميم (Design)	- وثيقة المتطلبات SRS	- التصميم المعماري للنظام - تصميم قاعدة البيانات - واجهات المستخدم	1. اتخاذ قرارات تصميم غير مثالية. 2. صعوبة موازنة الأداء مع التكلفة. 3. سوء توثيق التصميم مما يربك المطورين.
3. التطوير (Implementation / Coding)	- تصميم النظام المفصل	- الكود البرمجي الكامل - ملفات التهيئة والإعداد	1. أخطاء برمجية وصعوبات في الدمج. 2. ضعف التنسيق بين أعضاء الفريق. 3. تأخر الجدول الزمني بسبب التعقيد.
4. الاختبار (Testing)	- الكود المدمج - سيناريوهات الاختبار	- تقارير الاختبار - قائمة الأخطاء والمشكلات	1. نقص في تغطية الاختبارات. 2. ضغط الوقت قبل التسليم. 3. تجاهل بعض الحالات الحدّية.
5. النشر (Deployment)	- النظام المجرب والمستقر	- نسخة عاملة في بيئة الإنتاج - دليل المستخدم	1. مشاكل في توافق النظام مع البيئة الفعلية. 2. أخطاء أثناء النقل أو التثبيت. 3. الحاجة إلى تدريب المستخدمين.
6. الصيانة (Maintenance)	- النظام التشغيلي - ملاحظات المستخدمين	- تحديثات وإصلاحات - تحسينات على النظام	1. التعامل مع أخطاء غير متوقعة بعد الإطلاق. 2. تغيّر متطلبات العمل. 3. صعوبة تتبع الإصدارات وإدارة التحديثات.

المحور الثاني: الأدوار والمسؤوليات

1. جدول تفصيلي للأدوار المختلفة في فريق التطوير ومسؤولية كل دور في مراحل دورة حياة تطوير البرمجيات (SDLC)

المرحلة	محلل الأعمال (Business Analyst)	مدير المشروع (Project Manager)	المبرمج / المطوّر (Developer)	مهندس ضمان الجودة (QA / Tester)	مصمم الواجهة والتجربة (UI/UX Designer)	مسؤول الأنظمة / ديف أوبس (System Admin / DevOps)
تحليل المتطلبات	جمع وتحليل احتياجات المستخدمين، إعداد وثيقة المتطلبات (SRS).	تحديد نطاق المشروع والجدول الزمني وإدارة التواصل مع العميل.	تقديم تحليل الجدوى التقنية وتقدير الموارد.	مراجعة المتطلبات ووضع خطة اختبار مبدئية.	المساعدة في تصميم النماذج الأولية للواجهة.	التأكد من جاهزية البنية التحتية.
التصميم	مراجعة التصميم وضمان توافقه مع أهداف العمل.	تنظيم الاجتماعات ومتابعة التنفيذ.	تصميم النظام وقاعدة البيانات واختيار الأدوات.	إعداد حالات الاختبار التفصيلية.	تصميم واجهات المستخدم وتجربة الاستخدام.	إعداد بيئات التطوير والاختبار.
التطوير	المساعدة في توضيح المتطلبات أثناء التنفيذ.	الإشراف على سير العمل ومتابعة المراحل.	كتابة الكود وتنفيذه ودمج الوحدات.	تنفيذ اختبارات أولية وتسجيل الأخطاء.	دعم الفريق في العناصر البصرية والتصميمية.	إدارة بيئة التطوير والخوادم.
الاختبار	التأكد من توافق النظام مع متطلبات العميل.	متابعة نتائج الاختبارات وجدول الإصلاحات.	إصلاح الأخطاء وتحسين الأداء.	تنفيذ الاختبارات المختلفة وتوثيق النتائج.	اختبار واجهة المستخدم وتجربة الاستخدام.	دعم عملية الاختبار من الجانب التقني.
النشر	الموافقة النهائية على جاهزية النظام.	الإشراف على خطة النشر ومراقبة الإطلاق.	تجهيز ملفات التنفيذ والمشاركة في الإطلاق.	اختبار النسخة النهائية بعد الإطلاق.	التأكد من اتساق التصميم في بيئة الإنتاج.	تنفيذ عملية النشر ومراقبة النظام.
الصيانة	جمع ملاحظات المستخدمين واقتراح تحسينات.	إدارة التحديثات وتوزيع المهام.	إصلاح الأخطاء وإضافة المزايا الجديدة.	إعادة اختبار النظام بعد التحديث.	تحديث الواجهة عند إضافة خصائص جديدة.	تنفيذ النسخ الاحتياطية والتحديثات الأمنية.

2. كيف يتغير دور المبرمج خلال مراحل SDLC المختلفة

المرحلة	مهام المبرمج	وصف الدور

.مستشار تقني يقيّم قابلية التنفيذ	.مراجعة المتطلبات وتقديم الملاحظات التقنية	**تحليل المتطلبات**
.مصمم تقني يخطط للبنية البرمجية	.المساهمة في وضع الهيكل المعماري للنظام	**التصميم**
.الباني الفعلي للنظام	.كتابة الكود وتنفيذ الاختبارات الوحدوية	**التطوير**
.مصلح ومراجع للجودة	.إصلاح الأخطاء وتحسين الأداء	**الاختبار**
.داعم فني أثناء النشر	.إعداد ملفات النشر والمساعدة في الإطلاق	**النشر**
.المسؤول عن استمرارية وتطوير النظام	.تنفيذ التحديثات والإصلاحات المستقبلية	**الصيانة**

المحور الثالث: التطبيق العملي

1. مثال: تطبيق بنكي إلكتروني (Online Banking App)

التطبيق العملي	**المرحلة**
.تحديد احتياجات العملاء مثل التحويل المالي ودفع الفواتير، وجمع البيانات من خلال مقابلات واستبيانات	**تحليل المتطلبات**
.تصميم قاعدة البيانات والمعمارية الأمنية، وإنشاء واجهات استخدام سهلة وآمنة	**التصميم**
وربط Angular، أو React والواجهة الأمامية باستخدام Python أو Java برمجة الواجهة الخلفية بلغة .التطبيق بخدمات الدفع	**التطوير**
.إجراء اختبارات الأداء والأمان والتأكد من صحة العمليات المالية	**الاختبار**
.رفع التطبيق على خوادم الإنتاج وإطلاقه في متاجر الهواتف	**النشر**
.إضافة مزايا جديدة مثل البصمة وإصلاح الأخطاء وتحسين الأداء	**الصيانة**

2. الخطة الزمنية لمشروع برمجي مدته 6 أشهر

الأنشطة الرئيسية	**المدة**	**المرحلة**
.جمع وتحليل المتطلبات	أسابيع 3	**تحليل المتطلبات**
.إعداد التصاميم التقنية والواجهات	أسابيع 4	**التصميم**
.البرمجة والاختبارات الأولية	أسابيع 10	**التطوير**
.اختبارات وظيفية وتكاملية	أسابيع 4	**الاختبار**
.إطلاق النظام رسميًا	أسبوع واحد	**النشر**
.التحديثات والتحسينات	مستمرة بعد الإطلاق	**الصيانة**

.(**إجمالي المدة:** 6 أشهر (26 أسبوعًا تقريبًا

المحور الرابع: التحليل النقدي

1. تأثير تجاهل أو تنفيذ إحدى المراحل بشكل سيئ

مثال واقعي	النتائج السلبية	المرحلة
تطبيق توصيل لم يراع خاصية تتبع الطلبات، ففشل في تلبية احتياجات المستخدمين.	تنفيذ منتج غير مطابق لتوقعات العميل وزيادة التكاليف بسبب إعادة العمل.	**تحليل المتطلبات**
تطبيق بنكي واجه أخطاء في تسجيل الدخول بعد الإطلاق مما تسبب في فقدان ثقة العملاء.	ظهور أخطاء بعد الإطلاق تؤثر على سمعة الشركة وتؤدي إلى خسائر.	**الاختبار**

2. أهمية مرحلة الصيانة وأنواعها

لماذا تعتبر أطول وأكثر تكلفة؟

- بسبب الاستخدام المستمر للنظام وتغير احتياجات المستخدمين.
- تتطلب مراقبة وصيانة أمنية وتحديثات مستمرة.
- تمثل غالبًا **60% إلى 70% من التكلفة الكلية للمشروع**.

أنواع الصيانة:

1. **صيانة تصحيحية:** إصلاح الأخطاء بعد التشغيل.
2. **صيانة تكيفية:** تعديل النظام ليتوافق مع بيئات جديدة.
3. **صيانة تحسين الأداء:** تحسين الكفاءة وتجربة المستخدم.
4. **صيانة وقائية:** منع المشكلات المستقبلية عبر التحسين الداخلي.

المحور الخامس: الاتجاهات الحديثة

1. دور الذكاء الاصطناعي والأتمتة في تحسين دورة حياة تطوير البرمجيات (SDLC)

استخدامات الذكاء الاصطناعي أو الأتمتة	المرحلة
تحليل تعليقات المستخدمين آليًا واستخلاص المتطلبات من النصوص والمحادثات.	**تحليل المتطلبات**
توليد حالات اختبار تلقائية واكتشاف الأخطاء بشكل ذكي.	**الاختبار**
لاقتراح الأكواد ChatGPTو GitHub Copilot استخدام أدوات مساعدة مثل وتحسينها.	**التطوير**
التنبؤ بالأعطال قبل حدوثها عبر تحليل البيانات والأداء.	**الصيانة**

النتيجة:
يساهم الذكاء الاصطناعي والأتمتة في **تسريع العمل، تقليل الأخطاء البشرية، ورفع جودة المنتج البرمجي** في جميع مراحل SDLC.

1. What is the Software Development Life Cycle (SDLC)?

Definition:
The **Software Development Life Cycle (SDLC)** is a framework that describes the different **phases** involved in developing any software system or application — starting from **requirements gathering** to **deployment and maintenance**.
It represents a **systematic and organized plan** that ensures the development of high-quality software in an efficient, low-risk manner.

Importance of SDLC:

1. **Ensures product quality:** By following well-defined steps to verify that the software meets user requirements.
2. **Reduces costs and risks:** Early planning minimizes errors and post-deployment issues.
3. **Improves team communication:** Clear definition of roles and responsibilities in each phase.
4. **Facilitates project and time management:** Provides accurate scheduling and resource estimation.

5. **Predictable outcomes:** Helps monitor and evaluate project progress toward its goals.

Why SDLC is used in modern software projects:

- Modern software projects are **complex and large-scale**, requiring a structured approach.
- Development teams often include multiple roles (analysts, developers, testers...), so **methodical coordination** is essential.
- Continuous **changes in user and market needs** demand a clear process for system updates.
- It supports **modern methodologies** like Agile and DevOps within a clear organizational framework.

2. SDLC Phases: Inputs, Outputs, and Major Challenges

Phase	Main Inputs	Main Outputs	Key Challenges
1. Requirements Analysis	- Client request - Business documents - Interviews & surveys	- Software Requirements Specification (SRS) - Scope analysis	1. Difficulty understanding the exact user needs. 2. Changing requirements during the project. 3. Poor communication between analysts and users.
2. Design	- SRS document	- System architecture - Database design - User interface designs	1. Poor design decisions. 2. Balancing performance vs. cost. 3. Inadequate design documentation.
3. Development (Implementation / Coding)	- Detailed design documents	- Complete source code - Configuration files	1. Coding errors and integration issues. 2. Lack of coordination among team members. 3. Schedule delays due to complexity.

4. Testing	- Integrated code - Test scenarios	- Test reports - Bug and issue lists	1. Incomplete test coverage. 2. Time pressure before release. 3. Overlooking edge cases.
5. Deployment	- Fully tested and stable system	- Live production version - User manuals	1. Compatibility issues with production environment. 2. Deployment or installation errors. 3. Need for user training.
6. Maintenance	- Operational system - User feedback	- Updates and patches - System improvements	1. Unexpected post-launch issues. 2. Evolving business requirements. 3. Difficulty managing versions and updates.

Axis 2: Roles and Responsibilities

1. Detailed Table of Development Team Roles Across the SDLC

SDLC Phase	Business Analyst (BA)	Project Manager (PM)	Developer (Software Engineer)	Quality Assurance (QA / Tester)	UI/UX Designer	System Administrator / DevOps

Requirements Analysis	- Gather and analyze user and business needs. - Document functional and non-functional requirements. - Create SRS (Software Requirements Specification).	- Define project scope, budget, and timeline. - Manage communication between client and team. - Identify project risks.	- Provide technical feasibility analysis. - Estimate effort and resources. - Suggest alternative solutions.	- Review requirements for testability. - Prepare initial test plan.	- Support requirement visualization with mockups or flow diagrams.	- Validate infrastructure feasibility and constraints.
Design	- Validate that design supports business objectives. - Review use cases.	- Coordinate design reviews and schedule. - Control design changes.	- Define architecture and database structure. - Select frameworks and libraries.	- Review design for test planning. - Prepare detailed test cases.	- Design user interfaces, wireframes, and user experience flow.	- Set up development environments.
Development	- Clarify unclear requirements when needed.	- Track progress and resolve blockers. - Ensure deadlines are met.	- Write, test, and integrate code. - Conduct peer code reviews. - Perform unit testing.	- Execute initial testing cycles. - Identify and report bugs.	- Provide UI elements and support developers with design assets.	- Maintain servers for test/staging environments.
Testing	- Ensure business requirements are met.	- Manage defect tracking and testing	- Fix reported bugs. - Support	- Perform functional, integration, and regression	- Validate user interface consistency and usability.	- Support test deployment and rollback.

		timelines.	QA with test data.	testing. - Report test outcomes.		
Deployment	- Validate readiness for production.	- Approve release and manage launch plan.	- Support build creation and release scripts.	- Perform smoke tests post-deployment.	- Ensure visual consistency after deployment.	- Deploy the system to production and monitor performance.
Maintenance	- Collect feedback for improvements. - Suggest new features.	- Plan updates and manage version control.	- Fix issues, update code, add new modules.	- Re-test after fixes and updates. - Maintain regression testing.	- Update design for new features.	- Manage backups, security patches, and system monitoring.

2. How the Developer's Role Changes Across SDLC Phases

Phase	Developer's Tasks	Focus of the Role
Requirements Analysis	- Review and understand requirements. - Provide technical feedback.	Acts as **technical consultant** to ensure feasibility.
Design	- Contribute to system architecture and database design. - Choose programming tools and frameworks.	Acts as **solution designer** and system planner.
Development	- Write code and conduct unit testing. - Integrate modules.	Becomes the **main builder** of the system.
Testing	- Fix bugs reported by QA. - Refactor and optimize code.	Functions as **problem solver** ensuring quality.
Deployment	- Prepare deployment scripts. - Assist with production release.	Acts as **technical support engineer** during release.
Maintenance	- Implement new features. - Monitor system performance.	Serves as **system maintainer and improver**.

Axis 3: Practical Application

1. Example System: Online Banking Application

SDLC Phase	Example Implementation
Requirements Analysis	The bank identifies customer needs: fund transfer, bill payments, account statements, etc. Analysts gather input through interviews and surveys. Output: SRS document.
Design	System architects design the database for transactions, user authentication, and interfaces. UI designers create a simple and secure layout.
Development	Developers code backend APIs (Java, .NET, or Python) and frontend (React/Angular). Integration with payment gateways and authentication services.
Testing	QA performs functionality, security, and performance testing. Example: verifying the correct balance after transfers.
Deployment	The tested version is deployed to the production server. Mobile apps are uploaded to app stores.
Maintenance	Continuous updates such as adding biometric login or faster transaction processing. Bug fixes and performance enhancements.

2. Example Project Timeline (6 Months)

Phase	Duration	Key Activities
Requirements Analysis	3 weeks	Gather requirements, SRS documentation.
Design	4 weeks	System and UI design, architecture setup.
Development	10 weeks	Coding and internal unit testing.
Testing	4 weeks	Functional, integration, and user acceptance testing.
Deployment	1 week	Release and go-live.
Maintenance	Continuous (after deployment)	Fix issues and add new features.

Total project time: **6 months (26 weeks)**.

Axis 4: Critical Analysis

1. What Happens if a Phase Is Ignored or Done Poorly

Phase	Negative Impact	Example
Requirements Analysis Ignored	- Misunderstood needs → wrong product. - Increased rework and cost.	A delivery app developed without clear user flow might not allow users to track orders properly, leading to failure.
Testing Done Poorly	- Undetected bugs cause crashes and data loss. - Damage to company reputation.	In 2018, a banking app launched with login bugs that exposed customer data, causing loss of trust.

2. Maintenance Phase: Importance and Types

Why It's the Longest and Costliest Phase:

- The software is used daily by real users → continuous bug reports and feature requests.
- Requires ongoing monitoring, patching, and adaptation to new technologies.
- Accounts for **60–70% of total project cost** over its lifetime.

Types of Maintenance:

1. **Corrective Maintenance:** Fixing bugs and system errors after release.
2. **Adaptive Maintenance:** Updating the system for new operating systems or platforms.
3. **Perfective Maintenance:** Enhancing performance or usability.
4. **Preventive Maintenance:** Refactoring code to prevent future issues or security risks.

Axis 5: Modern Trends

1. Role of Artificial Intelligence (AI) and Automation in SDLC

SDLC Phase	How AI/Automation Improves It
Requirements Analysis	AI-driven tools can analyze user feedback and automatically extract requirements or identify missing needs from natural language documents.
Testing	AI-based test automation can generate and execute test cases automatically, detect anomalies, and predict high-risk areas for failure.
Development	AI-powered code assistants (e.g., GitHub Copilot, ChatGPT) help developers write, review, and optimize code faster.
Maintenance	Predictive analytics detect potential failures and performance bottlenecks before they happen.

Result:
AI and automation **reduce human error**, **accelerate delivery**, and **improve code quality** across multiple SDLC phases.